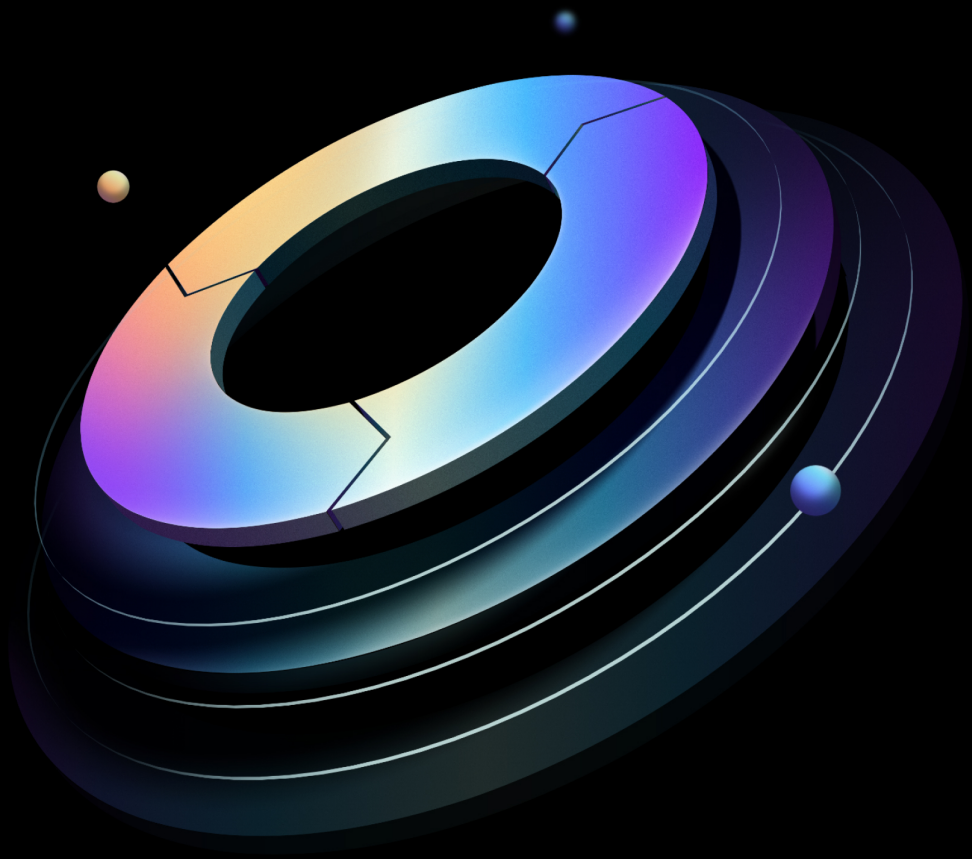
HashiCorp

Boundary Installation Guide

Built from commit d9ac71ead

HashiCorp | Validated Designs

27 August 2026



HashiCorp

Contents

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Audience

This document is primarily for practitioners (including platform, networking, identity, and information security teams) looking to deploy Boundary Enterprise clusters on-premises or in cloud infrastructure. After using this Installation Guide to deploy your Boundary Enterprise infrastructure, we recommend operators refer to the Boundary Administration Guide for day-2 operations and the Boundary User Guide to configure identity providers, secure user access, credential management, and other use cases.

1.3 Supported versions

This version of the guide relates to Boundary Enterprise 0.17.x+ent.

1.4 Language and definitions

This documentation intentionally uses technology agnostic terminology, however there are some terms which do not translate between the cloud providers. The following are the definitions of terms this document uses.

Term	Definition
Region	A physical location in a distinct geographic area containing one or more availability zones (AZ).
Availability zone (AZ)	An availability zone (AZ) is a single network failure domain that hosts part or all of a Boundary Enterprise cluster. Examples of availability zones include: an individual datacenter, an air-gapped rack in a datacenter, an availability zone in AWS or Azure, or a zone in GCP.
Public subnet	A network accessible by users.
Private subnet	A network used by applications and services, but not directly accessible by users.

1.5 Organizational requirements

We expect a single team, often referred to as the Platform Team, to handle the tasks in this guide. However, a successful Boundary Enterprise deployment requires collaboration across multiple organizational functions. The Platform Team needs to work with other teams, such as networking or security for tasks such as IP allocation or certificate management. If hosting infrastructure on a public cloud, consolidate these responsibilities within a unified cloud team. We recommend the platform team thoroughly review this Installation Guide to identify any teams they may rely on or need approval from for key deployment tasks. It is essential to designate a project lead to oversee the deployment process and ensure clear communication and coordination with all relevant teams and functions.

2 Architecture

2.1 Recommended deployment architecture

This section explores the recommended Boundary Enterprise architecture, which provides a highly available, scalable, and secure deployment suitable for production workloads.

The primary components that make up a Boundary Enterprise cluster are:

- Controller nodes
- Worker nodes
- Load balancer
- PostgreSQL database
- KMS (Key management service)

The following diagram shows the recommended architecture for deploying Boundary Enterprise within a single region.

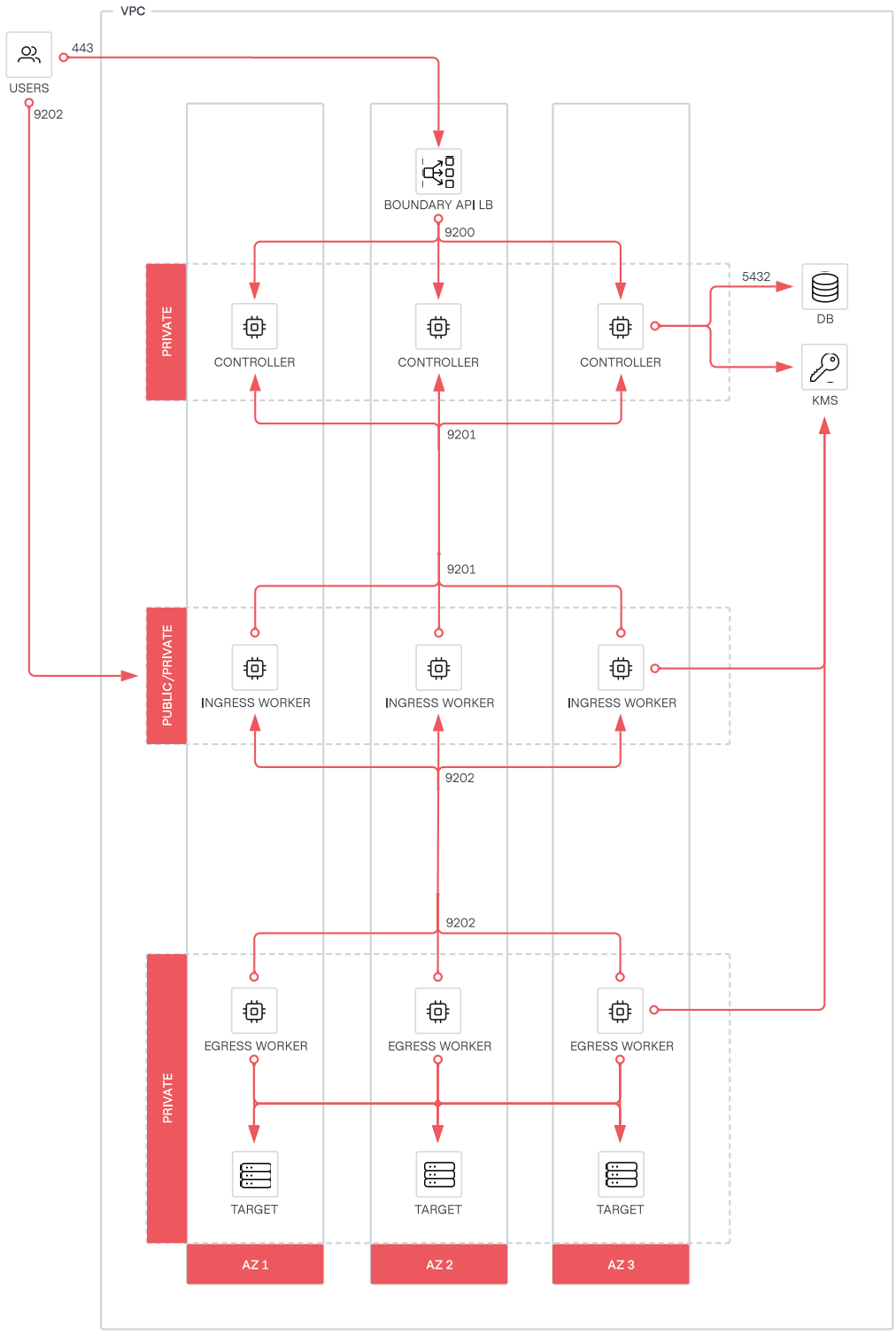


Figure 1: Boundary Single Region Deployment

2.2 Controllers

A minimum deployment consists of three controllers in three separate private subnets distributed across three availability zones to ensure high availability. Configure the controllers to run in a fault-tolerant setup, such as an auto-scaling group for a self-healing environment. Users authenticate with the controllers when using Boundary Enterprise. We recommend exposing the controller API and UI to your users through a layer 4 load balancer. See load balancing section for more information.

2.3 Database

Controllers are stateless, and manage all configuration through an external PostgreSQL database. We recommend configuring the PostgreSQL database for high availability. Please refer to the PostgreSQL high availability, load balancing, and replication documentation. If you use a managed service, refer to your provider's PostgreSQL high availability documentation.

2.4 Workers

A minimum deployment consists of at least three workers across different availability zones within each network boundary to ensure high availability. In environments which restrict inbound connections, deploy both ingress and egress workers to enable multi-hop session proxying, which only requires outbound connectivity. Please refer to the recommended architecture for more details.

2.5 Key management service

Boundary controllers use KMS keys to encrypt data at rest and in transit. Before the Boundary worker can proxy user sessions to targets, it must authenticate and register with the Boundary control plane. We recommend using a KMS-led authorization and authentication flow to auto-register the worker. This method requires the controller and worker to share a KMS key to authenticate the worker with the controller. We recommend using Vault's Transit secret engine for key management. If you use a managed service, refer to your provider's key management documentation for guidance.

2.6 Transport layer security

2.6.1 Client-to-controller TLS

We recommend configuring the TLS (with a public certificate, private key, and certificate authority bundle from a trusted certificate authority) on the Boundary controller nodes. Configure the load balancer to pass through TLS connections to controller nodes. Do not manage the TLS certificate on the load balancer. Terminating TLS connections at the controller nodes offers enhanced security by ensuring that a client request remains encrypted end-to-end. This reduces the attack surface for potential eavesdropping or data tampering.

2.6.2 Client-to-worker TLS

Workers do not require any configuration for their client-facing listeners. Instead, they create the TLS configuration dynamically using Server Name Indication during session authorization, and the session is then mutually authenticated.

2.6.3 Worker-to-upstream TLS

Workers establish TLS connections to upstream services (controllers or other workers) dynamically during registration. For more information, refer to this document.

2.7 Load balancing

Only the controller `api` listener requires a load balancer. The following table summarizes load balancing requirements by listener:

Component	Listener	Load balancer required
Controller	`api` (TCP 9200)	Yes
Controller	`cluster` (TCP 9201)	No
Worker	`proxy`	No
Worker	`cluster`	No

The `api` listener load balancer exposes the Boundary API and administrative console, and is the only load balancer in the recommended architecture. Use a layer 4 or 7 load balancer that polls the `/health` API endpoint to detect the node's status and direct traffic accordingly.

Do not place workers behind a shared load balancer. The Boundary control plane manages session distribution across available workers when clients initiate sessions to a target. Workers must be individually addressable. Refer to the worker-to-worker (multi-hop sessions) section for guidance on multi-hop deployments.

3 Detailed design

This section provides more architectural detail on each Boundary Enterprise component. Review this section to identify all technical and personnel requirements before moving to implementation.

3.1 Sizing

Every hosting environment is different, and every customer's Boundary Enterprise usage profile is different. Refer to the tables below for sizing recommendations for controller nodes and worker nodes, as well as small and large use cases, based on expected usage.

Small deployments would be appropriate for most initial production deployments or for development and testing environments. **Large** deployments are production environments with a consistently high workload, such as a large number of sessions.

3.1.1 Controller nodes

Size	CPU	Memory	Disk capacity	Network throughput
Small	2-4 core	8-16 GB RAM	50+ GB	Minimum 5 GB/s
Large	4-8 core	32-64 GB RAM	200+ GB	Minimum 10 GB/s

3.1.2 Worker nodes

Size	CPU	Memory	Disk capacity	Network throughput
Small	2-4 core	8-16 GB RAM	50+ GB	Minimum 10 GB/s
Large	4-8 core	32-64 GB RAM	200+ GB	Minimum 10 GB/s

Refer to Hardware sizing for Boundary servers for more details. These recommendations serve as a starting point for operations staff to observe and adjust to meet the unique needs of each deployment. Match your requirements and maximize the stability of your Boundary Enterprise controller and worker instances, by performing load tests and continue monitoring resource usage and all reported metrics from Boundary's telemetry.

3.1.3 Hardware considerations

CPU, memory, and storage performance requirements depend on your exact usage profile for example types of requests, average request rate, and peak request rate. Boundary Enterprise controllers and worker nodes have distinct resource needs as they handle different tasks. Refer to the Hardware Considerations for more information.

Enable audit logging in Boundary Enterprise. It is best to write audit logs to a separate disk for optimal performance. We recommend monitoring both the file descriptor usage and the memory consumption for each Boundary Enterprise worker node. These resources can become constrained depending on the number of clients connecting to Boundary Enterprise targets at any given time. If you have enabled session recording on a target, the worker stores the session recordings locally during the recording phase. Refer to Storage Considerations to determine how much storage to allocate for recordings on the worker nodes.

3.2 Networking

Network bandwidth requirements for Boundary Enterprise controllers and workers depend on your specific usage patterns. Consider bandwidth requirements for other external systems, such as monitoring and logging collectors. Refer to Network Considerations for more information. Monitor the networking metrics of Boundary Enterprise workers to prevent situations where they are unable to initiate session connections. Review your provider-specific virtual machine networking limitations. You must increase the VM size to achieve higher network throughput.

3.2.1 Network connectivity

Refer to Network Connectivity for the minimum requirements for Boundary Enterprise cluster nodes. You may also need to grant the Boundary Enterprise nodes outbound access to additional services that live elsewhere, either within your internal network or over the Internet. Examples may include:

- Authentication provider backends, such as Okta, Auth0, or Microsoft Entra ID
- Remote log handlers, such as a Splunk or ELK environment
- Metrics collection, such as Prometheus

3.3 Storage

Enable audit logging in Boundary Enterprise. For optimal performance, writing audit logs on a separate disk is advisable. The worker stores session recordings locally during the recording process, if enabled. When estimating worker storage needs, consider the number of concurrent sessions recorded on that worker. Refer to the storage guidelines to determine the appropriate amount of storage to allocate for recordings on the worker nodes.

3.4 KMS

Boundary Enterprise controllers and workers require different types of cryptographic keys. The KMS provider provides the root of trust for keys used for various purposes, such as protecting secrets, authenticating workers, recovering data, encrypting values in Boundary Enterprise's configuration. Refer to the KMS section for more information.

3.5 Traffic encryption

Boundary Enterprise is secure by default, and uses TLS for all network traffic communication. Boundary Enterprise has three types of connections, as described in the previous TLS section:

- Client-to-controller TLS
- Client-to-worker TLS
- Worker-to-upstream TLS

Refer to the TLS documentation for detailed information on each connection type.

From a load balancing requirement, always configure TLS pass-through. The load balancing section provides more information on this.

3.6 Load balancing

A layer 4 load balancer meets Boundary Enterprise's requirements. However, organizations may implement layer 7-capable load balancers for additional controls. Regardless of which, follow these requirements:

- HTTPS listener with valid TLS certificate for the domain it is serving or TLS pass-through
- Use TCP 9203 for health checks

Each major cloud provider offers one or more managed load-balancing services suitable for Boundary Enterprise. Follow the guidance provided in the load balancer recommendations.

3.6.1 Client-to-controller

Place Boundary Enterprise controller nodes in a private network and not exposed directly to the public Internet. Expose services such as the API and administrative console using a load balancer. This design utilizes a layer 4 load balancer with additional network security controls, such as security groups or firewall access control lists, to restrict the network flow to the load balancer interface.

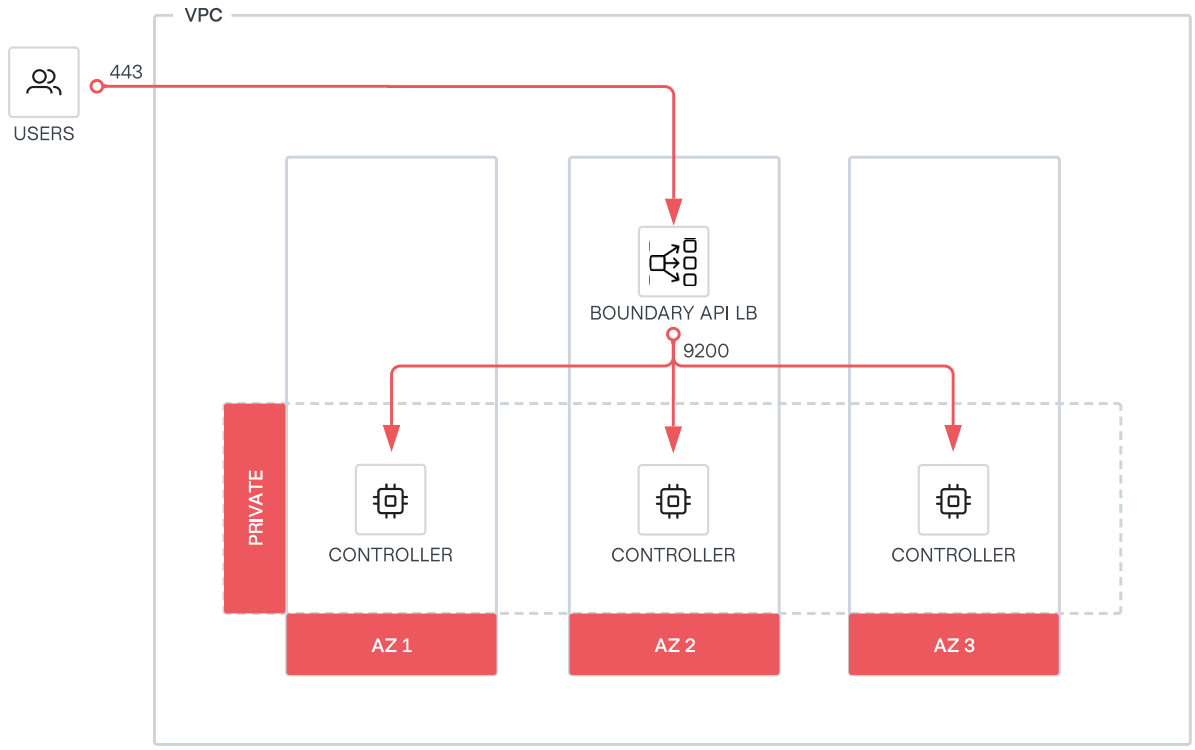


Figure 2: Boundary Enterprise client-to-controller

Health check

Use a load balancer to monitor the health of the Boundary Enterprise controller nodes. Do this by detecting the status of the `/health` endpoint.

This endpoint does not support any input. It returns an empty bodies to API responses.

Status	Description
200	GET `/health` returns HTTPS status `200` if the controller's API gRPC service is up.
5xx	GET `/health` returns HTTPS status `5xx` or request timeout if unhealthy.

Status	Description
503	GET `/health` returns HTTPS status `503` Service Unavailable status if the controller is shutting down.

Use the listener stanza to configure the controller's operational endpoints. By default, it listens on TCP 9203. The operational endpoint exposes both health and metrics endpoints.

Operational endpoint stanza configuration

```
# Ops listener for operations like health checks for load balancers
listener "tcp" {
  # Should be the address of the interface where your external systems' load balancer
      and/or metrics collectors etc. will connect on.
  address = "0.0.0.0:9203"
  # The purpose of this listener block
  purpose = "ops"

  tls_disable   = false
  tls_cert_file = "/etc/boundary.d/tls/boundary-cert.pem"
  tls_key_file  = "/etc/boundary.d/tls/boundary-key.pem"
}
```

3.6.2 Worker-to-controller

Workers connect directly to each Boundary Enterprise controller on TCP 9201 (the cluster port). A dedicated load balancer for worker-to-controller connectivity is not recommended.

For multi-cloud or multi-region deployments operating a single control plane, for example workers in another cloud or on-premises, it may be necessary to expose TCP 9201 externally so that remote workers can reach each controller directly. Each controller must be individually addressable on TCP 9201 — do not route this traffic through a shared load balancer. Assign each controller a stable IP address or DNS hostname to minimise configuration changes when controllers are replaced.

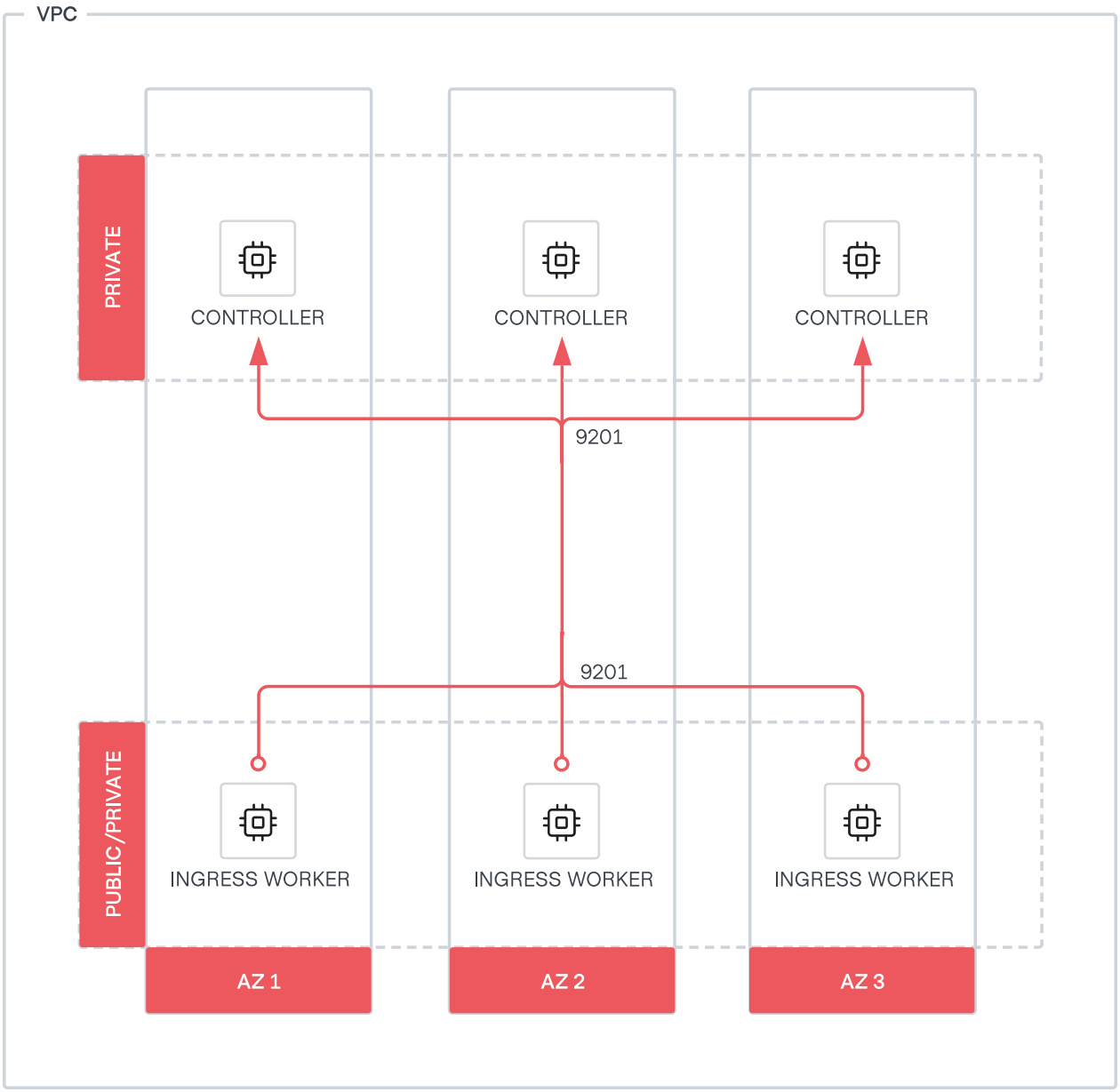


Figure 3: Boundary Enterprise Worker-to-controller

Use the `worker` stanza to configure `initial_upstreams` with the address of each controller. List each controller individually — do not use a shared load balancer address.

Worker stanza configuration

```
worker {
  # List each controller node individually by its cluster port address.
  # Do not use a shared load balancer address.
  initial_upstreams = [
    "controller-1.internal.example.com:9201",
    "controller-2.internal.example.com:9201",
    "controller-3.internal.example.com:9201",
  ]
}
```

3.6.3 Worker-to-worker (multi-hop sessions)

With multi-hop sessions, workers operate as intermediaries or egress workers. Do not place workers behind a shared load balancer. Each worker that has downstream workers must be individually addressable and must advertise its own address using the `public_addr` configuration field. The Boundary control plane manages session distribution across available workers when clients initiate sessions.

Configure `initial_upstreams` on each downstream worker with the explicit IP address or fully qualified domain name (FQDN) of each individual upstream worker. Do not set `initial_upstreams` to the address of a shared load balancer fronting multiple workers.

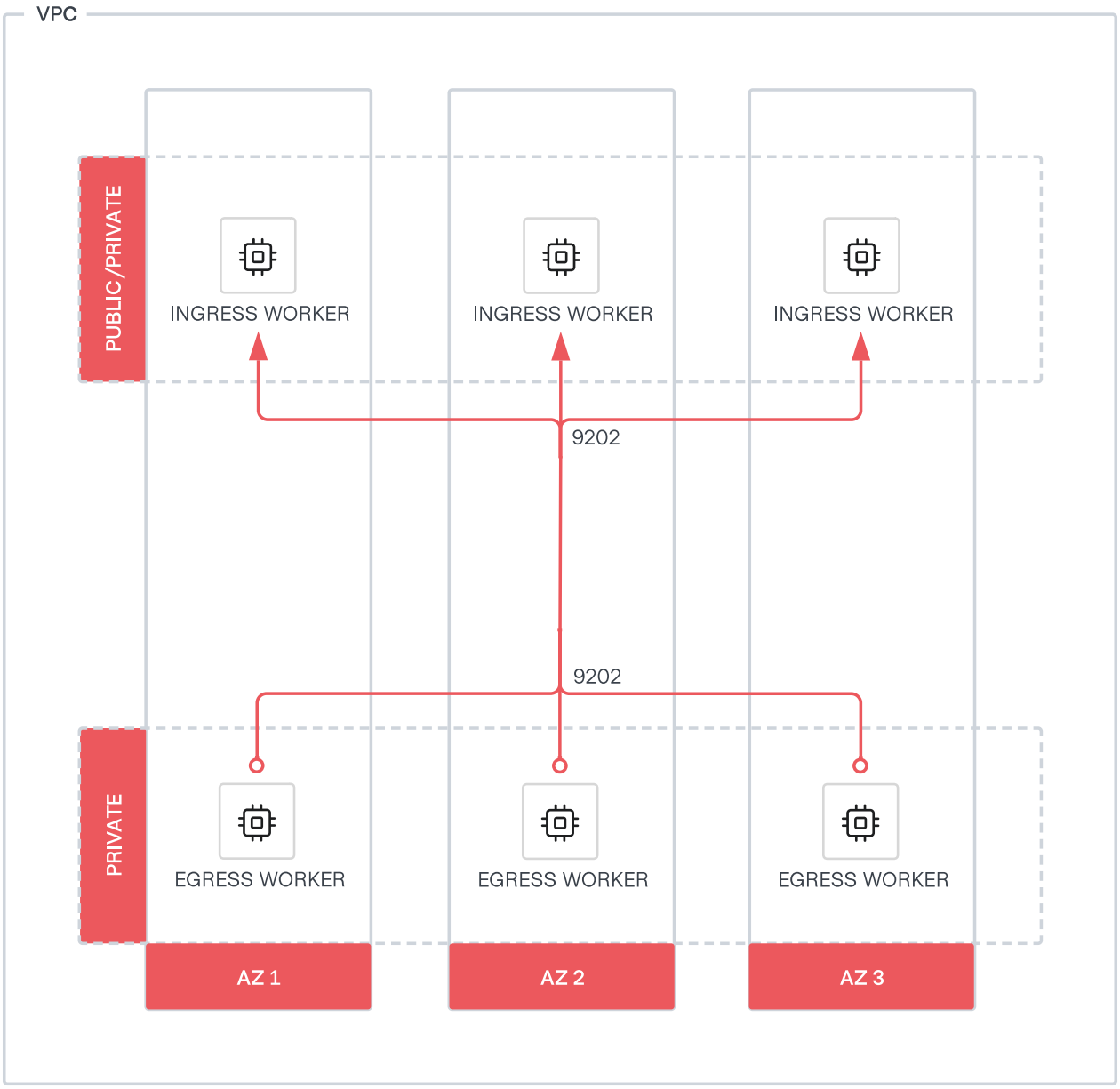


Figure 4: Boundary Enterprise Worker-to-worker

Use `public_addr` on each ingress worker so that it is individually addressable, and configure `initial_upstreams` on each egress worker to list each upstream worker explicitly.

Ingress worker stanza configuration

```
worker {
  # Advertise a stable, individually addressable address to downstream workers and
      clients.
  public_addr = "ingress-worker-1.internal.example.com"
  initial_upstreams = [
    "controller-1.internal.example.com:9201",
  ]
}
```

Egress worker stanza configuration

```
worker {
  # List each upstream (ingress) worker individually by address.
  # Do not use a shared load balancer address.
  initial_upstreams = [
    "ingress-worker-1.internal.example.com:9201",
  ]
}
```

When an upstream worker is replaced or a new worker is added, update the `initial_upstreams` list on the downstream workers that connect to it. Send a `SIGHUP` signal to the downstream worker process to reload the configuration without restarting it.

To minimise configuration changes when workers are recycled, consider assigning each upstream worker a static IP address or a stable DNS hostname.

3.7 Monitoring

Gaining visibility into Boundary Enterprise's controllers and workers is essential for production environments. It enables operators to manage, scale, and troubleshoot Boundary Enterprise efficiently and assists in detecting and mitigating anomalies in a deployment.

3.7.1 Logs

If events are not configured, for example using the events stanza, Boundary Enterprise outputs logs to stdout/stderr by default. Linux distributions typically capture Boundary Enterprise's log output to the system journal.

In production environments, use the events stanza to increase control over event logging. Event logging configured using the events stanza overrides the default behavior. For example, if configuring Boundary Enterprise to send events to a file, logs are no longer emitted to standard output or standard error.

Aggregate logs using a centralized platform for analysis, audit, and compliance, and aid in troubleshooting.

Minimum configuration

```
events {
  audit_enabled = true
  observations_enabled = true
  sysevents_enabled = true
  telemetry_enabled = false
  sink "stderr" {
    name = "all-events"
    description = "All events sent to stderr"
    event_types = ["*"]
    format = "hclog-text"
  }
}
```

3.7.2 Metrics

Metrics for controllers and workers are available for ingestion by third-party telemetry platforms, such as Prometheus and Grafana. The metrics use the OpenMetric exposition format. Refer to the Boundary Enterprise metrics documentation for a list of all available metrics.

Boundary Enterprise provides metrics through the /metrics path using a listener with the "ops" purpose.

Configure the controller's operational endpoints using the listener stanza. By default, it listens on TCP-9203. The operational `ops` listener exposes both health and metrics endpoints.

Operational endpoint stanza configuration

```
# Ops listener for operations like health checks for load balancers
listener "tcp" {
  # Should be the address of the interface where your external systems'
  # (eg: Load-Balancer and metrics collectors) will connect on.
  address = "0.0.0.0:9203"
  # The purpose of this listener block
  purpose = "ops"

  tls_disable   = false
  tls_cert_file = "/etc/boundary.d/tls/boundary-cert.pem"
  tls_key_file  = "/etc/boundary.d/tls/boundary-key.pem"
}
```

3.8 Failure considerations

Organizations must rely on the experience from their architecture, cloud, and platform teams to provide the appropriate levels of availability for Boundary Enterprise that meet their requirements. This architecture design leverages several principles and standard infrastructure services with major cloud providers to provide the highest availability and fault tolerance while balancing costs.

- **Auto scaling** to enhance fault tolerance. For example, if a Boundary Enterprise instance is unhealthy, the auto scaling service can replace it. You can also configure the service to use multiple availability zones and launch instances in another availability zone to compensate if one becomes unavailable.
- **Templating** images to decrease the time to deploy controllers and workers during the initial deployment, notably failure and scaling scenarios.
- **Infrastructure-as-code** to produce consistent, known deployments that reduce configuration errors.
- **Availability zones** to protect against data center failures. Spread Boundary Enterprise components across at least three availability zones in production environments. If deploying Boundary Enterprise to three availability zones is not possible, you can use the same architecture across one or two availability zones at the expense of a reliability risk in case of an availability zone outage.
- **Load balancing** to provide traffic redirection and health checking.

3.8.1 Controllers

Boundary Enterprise controllers are stateless. They store all state and configuration within PostgreSQL and can withstand failure scenarios where only one node is accessible. When a controller node fails, users are still be able to interact with other Boundary Enterprise controllers, assuming the presence of additional nodes behind a load balancer. Boundary Enterprise controllers depend on a PostgreSQL database. Ensure the database is reachable to all Boundary Enterprise controller nodes. It must also inherit the same levels of availability as the controllers. Do not deploy PostgreSQL on the controller nodes.

3.8.2 Workers

Boundary Enterprise uses workers as either proxies or reverse proxies. Workers routinely communicate with the controllers to report their health. In the event of a worker node failure, it is best practice to have at least three workers per network boundary and per type (ingress and egress) for production environments. Therefore, the controller assigns a user's proxy session to an active Boundary Enterprise worker node.

3.8.3 Availability zone failures

The following section provides recommendations for controllers and workers to overcome availability zone outages.

Controllers

By deploying Boundary Enterprise controllers in the recommended architecture across three availability zones with load balancing in front of them, the Boundary Enterprise control plane can survive outages in up to 2 availability zones.

To continue to serve Boundary Enterprise controller requests during a regional outage, a deployment like the one outlined in this guide must be in a different region. Use a multi-regional database technology to allow the nodes in the secondary region to communicate with the PostgreSQL database. For example, promote secondary region AWS RDS read replicas to read-write in the event the primary region fails.

Use services like AWS Global Accelerator for AWS, Cross-region Load Balancer for Azure, and GCP Cloud Load Balancer for GCP to load balance healthy Boundary Enterprise controller requests across regions.

Workers

We recommend deploying at least one worker per availability zone.

Should networking still be operational in an otherwise-failed availability zone, correct configuration of security rules allowing cross-subnet/AZ communication results in Boundary Enterprise proxying a user's session connection through a worker in another AZ.

If a Boundary Enterprise worker cannot reach its upstream worker or a controller, the user cannot establish a proxied session to the target.

4 Deployment

This section covers deploying Boundary Enterprise. Choose the deployment method that matches your environment:

- **Deploy with Terraform** — the recommended, automated path using the official HVD Terraform modules for AWS, Azure, or GCP.
- **Deploy manually** — a step-by-step manual installation of controllers and workers, suitable for private datacenters.

4.1 Deploy with Terraform

HashiCorp provides a set of official HVD modules to make it easier to deploy a Boundary Enterprise environment that adheres to the requirements and standards laid out in this HashiCorp Validated Design.

4.1.1 Platform-specific guidance

Deployment using Terraform modules

Select your cloud provider for the official HVD module links and platform-specific prerequisites.

AWS

HashiCorp Provides official HVD Modules to deploy Boundary Enterprise controllers and workers on AWS EC2.Before deployment, deploy the prerequisite infrastructure in AWS as below.1) A functional VPC with the required subnets. 2) A Boundary Enterprise license stored in AWS Secrets Manager. 3) A TLS private key and certificate, valid for the fully qualified domain name you plan to use with Boundary, that have been base64-encoded and uploaded to AWS Secrets Manager. 4) The ARN of the Boundary database password secret in AWS Secrets Manager.

Azure

HashiCorp provides official HVD Modules to deploy Boundary Enterprise controllers and workers on Azure VMs.Before deployment, deploy the prerequisite infrastructure in Azure.1) An Azure resource group. 2) An Azure Key Vault to store the prerequisite secret material. 3) A Boundary Enterprise license stored in Azure Key Vault. 4) A TLS private key and certificate, valid for the fully qualified domain name you plan to use with Boundary, that have been base64-encoded and uploaded to Azure Key Vault. 5) The name of the Boundary database password secret in Azure Key Vault.

GCP

HashiCorp provides official HVD Modules to deploy Boundary Enterprise controllers and workers on GCP GCE.Before deployment, deploy the prerequisite infrastructure in GCP.1) A function VPC with a public and private subnet. 2) A Google Secret Manager to store the prerequisite secret material. 3) A Boundary Enterprise license stored in Google Secret Manager. 4) A TLS private key and certificate, valid for the fully qualified domain name you plan to use with Boundary, that have been base64-encoded and uploaded to Google Secret Manager. 5) The version of the Boundary database password secret in Google Secret Manager.

Use these modules with Terraform to deploy a complete, end-to-end Boundary Enterprise deployment inside of your own cloud account.

While we have made efforts throughout this document to provide prescriptive best practices, we recognize that each organization has their own unique requirements and constraints when it comes to deploying infrastructure. Where feasible, we have included considerations needed when deploy-

ing Boundary in your cloud environment within the context of this Terraform module. The module contains additional capabilities that you may wish to review if the variables from this module do not suit your specific needs.

4.1.2 Deployment sequence overview

1. Deploy the prerequisite resources.
2. Obtain the license file.
3. Download the Boundary command-line tool (or the Boundary desktop client).
4. Download the Terraform command-line tool.
5. Obtain the HashiCorp Validated Design Terraform module for deploying Boundary Enterprise controllers and workers using the links for your CSP above.
6. Configure your cloud credentials.
7. Initialize your Terraform workspace.
8. Input your variables, including the values from your prerequisite deployment, in the module for deployment.
9. Create a Terraform plan for the controller.
10. Apply the plan.
11. Bootstrap the Boundary controller.
12. Create a Terraform plan for the worker(s).
13. Apply the plan.
14. Begin creating targets and using Boundary Enterprise.

4.1.3 Preparation

Create the certificate files

Create a standard X.509 certificate that to install on the Boundary servers. Refer to your organization's process on creating a new certificate that matches the DNS record you intend to direct users to when accessing Boundary.

Ensure the following files are available.

- The TLS certificate (tls-cert-secret.pub).
- The TLS private key (tls-cert-private.key).

- The CA bundle file from the certificate authority used to vend the certificate (tls-ca-bundle.pub).

Keep these files to hand, as they you need them later in the installation process.

Obtain the Boundary Enterprise license file

Obtain the Boundary Enterprise license file from your HashiCorp account team. This file contains a license key unique to your environment. Name the file something like *boundary.hclic*.

Keep this file to hand also, as you need it later in the installation process.

Download and install the Boundary command-line tool

- Download the appropriate package for your operating system from the HashiCorp Releases site.
- Unzip the package.
- Move the *boundary* binary (boundary.exe for Windows) to a directory in your system's *PATH*.
- *Optional*: Install Boundary Desktop client

Download and install the Terraform command-line tool

- Download the appropriate package for your operating system from the HashiCorp Releases site.
- Unzip the package.
- Move the Terraform binary (terraform.exe for Windows) to a directory in your system's *PATH*.

Download the Terraform module(s)

For the purpose of an automated deployment, Use these Terraform modules for your deployment, customizing where necessary. Once you have downloaded the module, navigate to the `examples/default/` directory. Use this as the base working directory during the installation process.

Configure cloud credentials

Select your cloud provider below for cloud credentials configuration guidance.

AWS

Ensure the correct AWS credentials are in place and accessible to Terraform. Terraform can read credentials from:* Credentials file: typically located at `$HOME/.aws/credentials` (`%UserProfile%\.aws\credentials` on Windows). * Environment variables as follows. - AWS_ACCESS_KEY_ID - AWS_SECRET_ACCESS_KEY - AWS_SESSION_TOKEN (if using an IAM role or other expiring credentials) - AWS_DEFAULT_REGIONFor complete details on how to configure AWS credentials for Terraform, see the HashiCorp Terraform AWS provider documentation.Ensure that the credentials have sufficient permissions to allow Terraform to perform the necessary actions.

Azure

Ensure the correct Azure credentials are in place and accessible to Terraform by running `az login` with your assumed credentials. For complete details on how to configure Azure credentials for Terraform, see the HashiCorp Terraform Azure provider documentation.Ensure the credentials have sufficient permissions to perform the necessary Terraform actions.

GCP

Ensure the correct GCP credentials are in place and accessible to Terraform. Terraform can read credentials from:* Credentials file: typically located at `$HOME/.config/gcloud/application_default_credentials.json` (`%APPDATA%\gcloud\application_default_credentials.json` on Windows). * Environment variables as follows. - `GOOGLE_CREDENTIALS`For complete details on how to configure GCP credentials for Terraform, see the HashiCorp Terraform GCP provider documentation.

Ensure the credentials have sufficient permissions to perform the necessary Terraform actions.

4.1.4 Installation

Initialize Terraform

Run `terraform init` to initialize your Terraform workspace and ensure there are no outstanding errors before continuing.

Configure variables for deployment

> **Warning**
>
> You can only configure variables for the installation module's `terraform.tfvars` file after all the prerequisite resources are available. You need to supply values from the prerequisites to the Vault module.

Review the `terraform.tfvars.example` file HashiCorp maintains in the `examples/default/` directory for explanations of each relevant variable. There is a `terraform.tfvars.example` file in the respective module for each public cloud provider. Copy this file to a file called `terraform.tfvars`, and then fill in the values for each declared variable with the applicable values for your environment.

Create and apply Terraform plan

From the `examples/default/` directory, generate a Terraform plan with the following command:

```
terraform plan -out=tfplan
```

Review the plan output, then apply the changes with this command:

```
terraform apply tfplan
```

Confirm the changes by typing `yes` when prompted.

Validate installation

After your `terraform apply` finishes, you can monitor the installation progress by connecting to your Boundary controller VM instance shell using SSH, AWS SSM, or Google IAP and observing the cloud-init (`user_data`) logs using the commands below in separate terminal windows.

```
tail -f /var/log/boundary-cloud-init.log
journalctl -xu cloud-final -f
```

> **Note**
>
> The `-f` argument is to follow the logs as they append in real time, and is optional. You may remove the `-f` for a static view.

The log files display the following message after the cloud-init (`user_data`) script finishes.

```
[INFO] boundary_custom_data script finished successfully!
```

Once the cloud-init script finishes, while still connected to the VM using SSH or equivalent, you can check the status of the Boundary service has the line below.

```
[INFO] boundary_custom_data script finished successfully!
```

From the terminal where you performed the `terraform apply` run the following command.

```
terraform output
```

Using the Terraform output command that references the load balancer name or IP address, create a new DNS entry that matches your TLS certificate and points to the load balancer for the **Vault** cluster. Set the following environment variable.

```
export BOUNDARY_ADDR="https://boundary.example.com"
```

Bootstrapping Boundary Enterprise

After deploying a Boundary Enterprise controller the system is in a partially initialized state. To complete initialization and configuration for initial authentication utilize the bootstrapping module.

Repeat the steps starting from the preceding *Initialize Terraform* section.

After bootstrapping is complete you can authenticate to the Boundary cluster using command-line tool or administrator UI.

Install Boundary workers

> **Note**
>
> Boundary workers in all instances require access to either the controller or the upstream workers. See the Network Connectivity page for more information.

Utilize the Boundary Enterprise worker HVD module for AWS, Azure, or GCP and repeat the steps starting from *Initialize Terraform* for this new module.

After your `terraform apply` finishes, monitor the installation progress by connecting to your Boundary worker VM instance shell using SSH, AWS SSM, or Google IAP and observing the cloud-init (user_data) logs using the commands below in different terminal windows.

```
tail -f /var/log/boundary-cloud-init.log
journalctl -xu cloud-final -f
```

> **Note**
>
> The `-f` argument is to follow the logs as they append in real time, and is optional. You may remove the `-f` for a static view.

The log files display the following message after the cloud-init (user_data) script finishes.

```
[INFO] boundary_custom_data script finished successfully!
```

Once the cloud-init script finishes, while still connected to the VM using SSH you can check the status of the boundary service using the command below.

```
sudo systemctl status boundary
```

After the Boundary worker has deployed, it is visible in the Boundary clusters' workers list.

4.2 Deploy manually

4.2.1 Overview

This section details the steps to create a Boundary Enterprise cluster manually in a private datacenter. This guide assumes that you have already read the Recommended Deployment Architecture and Detailed Design section of this guide and have a basic understanding of the Boundary Enterprise architecture and the steps required to deploy it in a private datacenter.

This guide uses the following names and IP addresses for the Boundary Enterprise controllers and workers.

DNS Name	IP Address	Node Type	Location
controller-api-lb.boundary.domain	controller_api_lb_addr	Internet-facing controller API load balancer (TCP 443)	(all zones)
controller-cluster-lb.boundary.domain	controller_cluster_lb_address	Internal-facing controller cluster load balancer (TCP 9201)	(all zones)
controller1.boundary.do	10.0.253.11	Controller VM	zone1
controller2.boundary.domain	10.0.254.12	Controller VM	zone2
controller3.boundary.do	10.0.255.13	Controller VM	zone3
ingressworker-lb.boundary.domain	ingress_lb_address	Internal-facing ingress worker load balancer (TCP 9202)	(all zones)
ingressworker1.boundar	10.0.253.101	Ingress worker VM	zone1
ingressworker2.boundary.domain	10.0.254.102	Ingress worker VM	zone2
ingressworker3.boundar	10.0.255.103	Ingress worker VM	zone3
egressworker1.boundary.domain	10.0.253.201	Egress worker VM	zone1
egressworker2.boundary	10.0.254.202	Egress worker VM	zone2
egressworker3.boundary.domain	10.0.255.203	Egress worker VM	zone3

4.2.2 Prepare

License

Obtain your active Boundary Enterprise license file. If you do not have this file, contact your HashiCorp account team.

Servers

1. Refer to the Detailed Design section of the guide for more information on what to consider the sizing based on your environment.
2. Identify the availability zones within your datacenter where you plan to host your Boundary controllers and ingress/egress. For the rest of this document, we refer to these as zone1, zone2, zone3, and so on.
3. Build the servers. Ensure there is 1 Boundary Enterprise controller, ingress/egress worker per each of the 3 availability zones, for a total of 3 servers. For the rest of this document, we refer to these servers as controller1, controller2, controller3, ingressworker1, ingressworker2, ingressworker3, egressworker1, egressworker2, egressworker2, and so on.
4. Ensure you can log in to each server as a user with sudo or root privileges using SSH or equivalent.

Load balancer

1. A layer 4 load balancer exposes controller API and administrator UI using HTTPS (port 443) to Boundary Enterprise clients. The load balancer distributes Boundary Enterprise client requests to the controller's API port (default TCP 9200).
2. Another layer 4 load balancer exposes the controller's cluster port (default TCP 9201) for workers session authorization, credentials, and so on.
3. Refer to the Detailed Design section of the guide for more information on how to configure the load balancer.

PostgreSQL

Controllers are stateless and manage all configurations through an external PostgreSQL database. We recommend configuring the PostgreSQL database for high availability. Please refer to PostgreSQL High Availability, Load Balancing, and Replication. If you use a managed service, refer to your provider's PostgreSQL high availability documentation.

Storage for session recording

We recommend using S3-compliant Object Storage for audit logging and session recording. Refer to the Detailed Design section of the guide for more information on how to configure storage for session recording.

4.2.3 Boundary Enterprise controller configuration

TLS

Create an X.509 certificate. Install this onto each of the Boundary Enterprise controllers. Refer to your organization's process on creating a new certificate that matches the DNS record you intend to direct users to when accessing Boundary Enterprise. In this case, the DNS record pointing at the load balancer: `boundary.domain`. Please replace `boundary.domain` with your actual domain name.

You need these files:

1. The certificate (`cert.pem`).
2. The certificate's private key (`key.pem`).
3. The certificate authority bundle from a trusted certificate authority (`bundle.pem`).

You may have to create a new directory to store the certificate material at `/etc/boundary.d/tls`.

```
ls -l /etc/boundary.d/tls
-rw-r----- 1 boundary boundary 1801 Oct 17 03:47 bundle.pem
-rw-r----- 1 boundary boundary 1842 Oct 17 03:47 cert.pem
-rw-r----- 1 boundary boundary 1679 Oct 17 03:47 key.pem
```

Specify the following configuration in the `/etc/boundary.d/controller.hcl` file to contain the TLS certificates configuration.

```
# API listener configuration block
listener "tcp" {
  address              = "0.0.0.0:9200"
  purpose              = "api"
  tls_disable          = false
  tls_cert_file        = "/etc/boundary.d/tls/cert.pem"
  tls_key_file         = "/etc/boundary.d/tls/key.pem"
  tls_client_ca_file   = "/etc/boundary.d/tls/bundle.pem"
  cors_enabled         = true
  cors_allowed_origins = ["*"]
}

# Ops listener for operations like health checks for load balancers
listener "tcp" {
  address            = "0.0.0.0:9203"
  purpose            = "ops"
  tls_disable        = false
  tls_cert_file      = "/etc/boundary.d/tls/cert.pem"
  tls_key_file       = "/etc/boundary.d/tls/key.pem"
  tls_client_ca_file = "/etc/boundary.d/tls/bundle.pem"
}
```

Note

We recommend storing the TLS material and license key for Boundary Enterprise controllers in HashiCorp Vault, HCP Vault Secrets or cloud provider equivalents.

KMS for controllers

You need four KMS keys:

1. root
    - The root key is the primary encryption key used by Boundary Enterprise.
2. worker-auth
    - The controller and worker share this to authenticate a worker to the controller.
3. recovery
    - Use this key for rescue/recovery operations in case of system issues or when normal authentication methods are unavailable.

4. bsr
    - Use this key for session recording capabilities. Recording of session data uses this key and ensures the integrity of those recordings.

Specify the following configuration in the `/etc/boundary.d/controller.hcl` file to contain the KMS keys configuration.

```
# Root KMS Key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "root"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey1"
}

# Recovery KMS Key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "recovery"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey2"
}

# Worker-Auth KMS Key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "worker-auth"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey3"
}

# BSR KMS Key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "bsr"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey4"
}
```

Refer to the Vault Transit page to configure Boundary Enterprise to use Vault's Transit secret engine for key management. If you use a managed service, refer to our KMS documentation for guidance.

Prepare controller configuration to initialize a PostgreSQL database

Use the following in the `/etc/boundary.d/controller.hcl` file to configure the database URL.

```
# Controller configuration block
controller {
  name        = "<controller1>"           # update here for other controllers
  description = "<Boundary Controller 1>" # update here for other controllers
  database {
    url = "postgresql://POSTGRESQL_CONNECTION_STRING"
  }

  license = "file:////opt/boundary/license/license.hclic"
}
```

Prepare Boundary Enterprise controller configuration

Populate the `/etc/boundary.d/controller.hcl` file with the configuration information below.

Use the following controller configuration on each controller node, replacing the name, description, database URL, license path, listener IP address and KMS configuration in each case.

```
# disable memory from being swapped to disk
disable_mlock = true
telemetry {
  prometheus_retention_time = "24h"
  disable_hostname          = true
}

# Controller configuration block
controller {
  name        = "<controller1>"                     # update here for other
      controllers
  description = "<Boundary Enterprise Controller 1>" # update here for other
      controllers
  database {
    url = "postgresql://POSTGRESQL_CONNECTION_STRING"
  }

  license = "file:////opt/boundary/license/license.hclic"
}

# API listener configuration block
listener "tcp" {
  address              = "0.0.0.0:9200"
  purpose              = "api"
  tls_disable          = false
  tls_cert_file        = "/etc/boundary.d/tls/cert.pem"
  tls_key_file         = "/etc/boundary.d/tls/key.pem"
  tls_client_ca_file   = "/etc/boundary.d/tls/bundle.pem"
  cors_enabled         = true
  cors_allowed_origins = ["*"]
}

# Data-plane listener configuration block (used for worker coordination)
listener "tcp" {
  address = "<10.0.253.11>:9201" # update here for other controllers IP address
  purpose = "cluster"
}

# Ops listener for operations like health checks for load balancers
listener "tcp" {
  address            = "0.0.0.0:9203"
  purpose            = "ops"
  tls_disable        = false
  tls_cert_file      = "/etc/boundary.d/tls/cert.pem"
  tls_key_file       = "/etc/boundary.d/tls/key.pem"
  tls_client_ca_file = "/etc/boundary.d/tls/bundle.pem"
}

# Events (logging) configuration. This configured logging for ALL events to both
# stderr and a file at /var/log/boundary/controller.log
events {
  audit_enabled       = true
  sysevents_enabled   = true
  observations_enable = true
  sink "stderr" {
    name = "all-events"
    description = "All events sent to stderr"
```

Initialize a PostgreSQL database

Before you can start Boundary Enterprise, you must initialize the database from one controller.

The following command initializes a Boundary Enterprise's database with the configuration specified in the `/etc/boundary.d/controller.hcl` file:

```
boundary database init -config=/etc/boundary/controller.hcl
```

Starting Boundary Enterprise controller service

When the configuration files are in place on each Boundary Enterprise controller, you can proceed to enable and start the binary using `systemd` on each of the Boundary Enterprise controller nodes.

Perform these steps on all Boundary Enterprise controllers:

1. Create a `boundary` user, and create directories for Boundary Enterprise configuration owned by this user.

```
adduser --system --group boundary || true
mkdir -p /etc/boundary.d /etc/boundary.d/tls /opt/boundary/license /var/log/boundary
chown -R boundary:boundary /etc/boundary.d /var/log/boundary
```

1. Download the Boundary Enterprise package from HashiCorp. Unzip the package and move the `boundary` binary to a shared `PATH` location such as `/usr/local/bin`, owned by the `boundary` user.

```
curl -O https://releases.hashicorp.com/boundary/0.17.1+ent/boundary_0.17.1+
    ent_linux_amd64.zip
unzip boundary_0.17.1+ent_linux_amd64.zip
mv boundary /usr/local/bin/
chown boundary:boundary /usr/local/bin/boundary
```

1. Add the license file, certificate files, and relevant configuration file to `/etc/boundary.d`. In the end, the directory looks like the below.

```
$ chown boundary:boundary /etc/boundary.d/*
$ chmod 640 /etc/boundary.d/*
$ ls -l /etc/boundary.d/
-rw-r-----  1 boundary boundary 1652 Oct 17 03:47 controller.hcl
drwxr-x---  2 boundary boundary 4096 Oct 17 03:47 tls

$ ls -l /etc/boundary.d/tls
-rw-r----- 1 boundary boundary 1801 Oct 17 03:47 bundle.pem
-rw-r----- 1 boundary boundary 1842 Oct 17 03:47 cert.pem
-rw-r----- 1 boundary boundary 1679 Oct 17 03:47 key.pem

$ ls -l /opt/boundary/license
-rw-rw-r-- 1 root     root     3514 Aug 15 18:21 EULA.txt
-rw-rw-r-- 1 root     root     4922 Aug 15 18:21 LICENSE.txt
-rw-rw-r-- 1 root     root     9518 Aug 15 18:21 TermsOfEvaluation.txt
-rw-r--r-- 1 root     root     1163 Oct 17 03:47 license.hclic

$ ls -la /var/log/boundary
total 8
drwxr-x---  2 boundary boundary 4096 Oct 17 06:23 .
drwxrwxr-x 11 root     syslog   4096 Oct 17 06:23 ..
```

1. Create a `systemd` unit file for the Boundary Enterprise service, then load it into `systemd`. Note that the `ExecStart` line runs the `boundary` binary pointing to your `controller.hcl` file.

```
$ cat << EOF >> /etc/systemd/system/boundary.service
[Unit]
Description="HashiCorp Boundary Enterprise"
Documentation=https://developer.hashicorp.com/boundary/docs
Requires=network-online.target
After=network-online.target
ConditionFileNotEmpty=/etc/boundary.d/controller.hcl
StartLimitIntervalSec=60
StartLimitBurst=3

[Service]
User=boundary
Group=boundary
ProtectSystem=full
ProtectHome=read-only
PrivateTmp=yes
PrivateDevices=yes
SecureBits=keep-caps
AmbientCapabilities=CAP_IPC_LOCK
CapabilityBoundingSet=CAP_SYSLOG CAP_IPC_LOCK
NoNewPrivileges=yes
ExecStart=/usr/bin/boundary server -config=/etc/boundary.d/controller.hcl
ExecReload=/bin/kill --signal HUP $MAINPID
KillMode=process
KillSignal=SIGINT
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
LimitNOFILE=65536
LimitMEMLOCK=infinity

[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl daemon-reload
systemctl enable boundary
```

1. Start the Boundary Enterprise controller service.

```
systemctl start boundary
```

4.2.4 Boundary Enterprise ingress workers configuration

KMS for ingress workers

Get the `worker-auth` KMS key from the KMS for controllers section. This key enables secure communication between workers and controllers, ensuring that only authorized workers can connect to each other.

Specify the following configuration in the `/etc/boundary.d/worker.hcl` file to contain the KMS keys configuration.

```
# Worker-auth KMS key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "worker-auth"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey3"
}
```

Prepare ingress workers configuration

Populate the `/etc/boundary.d/worker.hcl` file with the configuration information below replacing content prompted in angled brackets with the correct characters for your deployment.

All three ingress workers' configurations are the same except the `worker configuration stanza`
.

```
# disable memory from being swapped to disk
disable_mlock = true
telemetry {
  prometheus_retention_time = "24h"
  disable_hostname          = true
}

# worker block for configuring the specifics of the worker service
worker {
  public_addr = "<10.0.253.101>"      # update here for other ingress workers ip
      address
  name = "<ingressworker1>"           # update here for other ingress workers name
  initial_upstreams = ["<controller_cluster_lb_address>:9201"]
  recording_storage_path="/opt/boundary/bsr"
  recording_storage_minimum_available_capacity="500MB"
  tags {"app"="worker","env"="uat","bsr"="enabled","worker-type"="ingress"}
}

# listener denoting this is a worker proxy
listener "tcp" {
  address            = "0.0.0.0:9202"
  purpose            = "proxy"
}

# Ops listener for operations like health checks for ingress workers
listener "tcp" {
  address     = "0.0.0.0:9203"
  purpose     = "ops"
  tls_disable = true
}

# Events (logging) configuration. This
# configured logging for ALL events to both
# stderr and a file at /var/log/boundary/ingress-worker.log
events {
  audit_enabled       = true
  sysevents_enabled   = true
  observations_enable = true
  sink "stderr" {
    name = "all-events"
    description = "All events sent to stderr"
    event_types = ["*"]
    format = "cloudevents-json"
  }
  sink {
    name = "file-sink"
    description = "All events sent to a file"
    event_types = ["*"]
    format = "cloudevents-json"
    file {
      path = "/var/log/boundary"
      file_name = "ingress-worker.log"
    }
    audit_config {
      audit_filter_overrides {
        sensitive = "redact"
        secret    = "redact"
```

Starting Boundary Enterprise ingress worker service

When the configuration files are in place on each Boundary Enterprise ingress worker, you can proceed to enable and start the binary using `systemd` on each of the Boundary Enterprise ingress worker nodes.

Perform these steps on all Boundary Enterprise ingress workers:

1. Create a `boundary` user, and create directories for Boundary Enterprise configuration owned by this user.

```
adduser --system --group boundary || true
mkdir -p /etc/boundary.d /opt/boundary/bsr /var/log/boundary
chown -R boundary:boundary /etc/boundary.d /opt/boundary/bsr /var/log/boundary
```

1. Use the official well-architected method to download the Boundary Enterprise binary and signature files from HashiCorp and confirm the integrity using the `gpg` binary. Unzip the package and move the `boundary` binary to a shared `PATH` location such as `/usr/local/bin`, owned by the `boundary` user as per the below.

```
unzip boundary_0.17.1+ent_linux_amd64.zip
mv boundary /usr/local/bin/
chown boundary:boundary /usr/local/bin/boundary
```

1. Add relevant configuration file to `/etc/boundary.d`. In the end, the directory looks like this:

```
$ chown boundary:boundary /etc/boundary.d/*
$ chmod 640 /etc/boundary.d/*
$ ls -l /etc/boundary.d/
-rw-r-----  1 boundary boundary  704 Oct 17 06:23 worker.hcl

$ ls -l /opt/boundary/
drwxr-x--- 3 boundary boundary 4096 Oct 17 06:23 bsr
drwxr-x--- 2 boundary boundary 4096 Oct 17 06:23 data

$ ls -la /var/log/boundary
total 8
drwxr-x---  2 boundary boundary 4096 Oct 17 06:23 .
drwxrwxr-x 11 root     syslog   4096 Oct 17 06:23 ..
```

1. Create a systemd unit file for the Boundary Enterprise service, then load it into `systemd`. Note that the `ExecStart` line runs the `boundary` binary pointing to your `worker.hcl` file:

```
$ cat << EOF >> /etc/systemd/system/boundary.service
[Unit]
Description="HashiCorp Boundary Enterprise"
Documentation=https://www.boundaryproject.io/docs/
Requires=network-online.target
After=network-online.target
ConditionFileNotEmpty=/etc/boundary.d/worker.hcl
StartLimitIntervalSec=60
StartLimitBurst=3

[Service]
User=boundary
Group=boundary
ProtectSystem=full
ProtectHome=read-only
PrivateTmp=yes
PrivateDevices=yes
SecureBits=keep-caps
AmbientCapabilities=CAP_IPC_LOCK
CapabilityBoundingSet=CAP_SYSLOG CAP_IPC_LOCK
NoNewPrivileges=yes
ExecStart=/usr/bin/boundary server -config=/etc/boundary.d/worker.hcl
ExecReload=/bin/kill --signal HUP $MAINPID
KillMode=process
KillSignal=SIGINT
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
LimitNOFILE=65536
LimitMEMLOCK=infinity

[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl daemon-reload
systemctl enable boundary
```

1. Start the Boundary Enterprise ingress worker service.

```
systemctl start boundary
```

4.2.5 Boundary Enterprise egress workers configuration

KMS for egress workers

Get the `worker-auth` KMS key from the KMS for controllers section. This key enables secure communication between workers and controllers, ensuring that only authorized workers can connect to each other.

Use the following KMS configuration block in the `/etc/boundary.d/worker.hcl` file.

```
# Worker-Auth KMS Key (managed by AWS KMS in this example)
kms "awskms" {
  purpose    = "worker-auth"
  region     = "ap-southeast-1"
  kms_key_id = "abcd1234-a123-456a-a12b-aexamplekey3"
}
```

Prepare egress workers configuration

Populate the `/etc/boundary.d/worker.hcl` file with the configuration information below.

The configuration of all three egress workers must be identical except the `worker` and `kms` blocks. Amend these as necessary.

```
# disable memory from being swapped to disk
disable_mlock = true
telemetry {
  prometheus_retention_time = "24h"
  disable_hostname          = true
}

# worker block for configuring the specifics of the worker service
worker {
  public_addr = "<10.0.253.201>" # update here for other egress workers IP address
  name = "<egressworker1>"       # update here for other egress workers name
  initial_upstreams = ["<ingress_lb_address>:9202"]
  recording_storage_path="/opt/boundary/bsr"
  recording_storage_minimum_available_capacity="500MB"
  tags {"app"="worker","env"="uat","bsr"="enabled","worker-type"="egress"}
}

# listener denoting this is a worker proxy
listener "tcp" {
  address            = "0.0.0.0:9202"
  purpose            = "proxy"
}

# Ops listener for operations like health checks for ingress workers
listener "tcp" {
  address     = "0.0.0.0:9203"
  purpose     = "ops"
  tls_disable = true
}

# Events (logging) configuration. This
# configured logging for ALL events to both
# stderr and a file at /var/log/boundary/egress-worker.log
events {
  audit_enabled       = true
  sysevents_enabled   = true
  observations_enable = true
  sink "stderr" {
    name = "all-events"
    description = "All events sent to stderr"
    event_types = ["*"]
    format = "cloudevents-json"
  }
  sink {
    name = "file-sink"
    description = "All events sent to a file"
    event_types = ["*"]
    format = "cloudevents-json"
    file {
      path = "/var/log/boundary"
      file_name = "egress-worker.log"
    }
    audit_config {
      audit_filter_overrides {
        sensitive = "redact"
        secret    = "redact"
      }
```

Starting Boundary Enterprise egress worker service

When the configuration files are in place on each Boundary Enterprise egress worker, enable the binary using `systemd` on each of the Boundary Enterprise egress worker nodes.

Perform these steps on all Boundary Enterprise egress workers:

1. Create a `boundary` user, and create directories for Boundary Enterprise configuration owned by this user.

```
adduser --system --group boundary || true
mkdir -p /etc/boundary.d /opt/boundary/bsr /var/log/boundary
chown -R boundary:boundary /etc/boundary.d /opt/boundary/bsr /var/log/boundary
```

1. Use the official well-architected method to download the Boundary Enterprise binary and signature files from HashiCorp and confirm the integrity using the `gpg` binary. Unzip the package and move the `boundary` binary to a shared `PATH` location such as `/usr/local/bin`, owned by the `boundary` user as per the below.

```
unzip boundary_0.17.1+ent_linux_amd64.zip
mv boundary /usr/local/bin/
chown boundary:boundary /usr/local/bin/boundary
```

1. Add relevant configuration file to `/etc/boundary.d`. In the end, the directory looks like this:

```
$ chown boundary:boundary /etc/boundary.d/*
$ chmod 640 /etc/boundary.d/*
$ ls -l /etc/boundary.d/
-rw-r-----  1 boundary boundary  704 Oct 17 06:23 worker.hcl

$ ls -l /opt/boundary/
drwxr-x--- 3 boundary boundary 4096 Oct 17 06:23 bsr
drwxr-x--- 2 boundary boundary 4096 Oct 17 06:23 data

$ ls -la /var/log/boundary
total 8
drwxr-x---  2 boundary boundary 4096 Oct 17 06:23 .
drwxrwxr-x 11 root     syslog   4096 Oct 17 06:23 ..
```

1. Create a `systemd` unit file for the Boundary Enterprise service, then load it into `systemd`. Note that the `ExecStart` line runs the `boundary` binary pointing to your `worker.hcl` file:

```
$ cat << EOF >> /etc/systemd/system/boundary.service
[Unit]
Description="HashiCorp Boundary Enterprise"
Documentation=https://www.boundaryproject.io/docs/
Requires=network-online.target
After=network-online.target
ConditionFileNotEmpty=/etc/boundary.d/worker.hcl
StartLimitIntervalSec=60
StartLimitBurst=3

[Service]
User=boundary
Group=boundary
ProtectSystem=full
ProtectHome=read-only
PrivateTmp=yes
PrivateDevices=yes
SecureBits=keep-caps
AmbientCapabilities=CAP_IPC_LOCK
CapabilityBoundingSet=CAP_SYSLOG CAP_IPC_LOCK
NoNewPrivileges=yes
ExecStart=/usr/bin/boundary server -config=/etc/boundary.d/worker.hcl
ExecReload=/bin/kill --signal HUP $MAINPID
KillMode=process
KillSignal=SIGINT
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
LimitNOFILE=65536
LimitMEMLOCK=infinity

[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl daemon-reload
systemctl enable boundary
```

1. Start the Boundary Enterprise egress worker service.

```
systemctl start boundary
```

4.2.6 Next steps

After setting up a Boundary Enterprise cluster, it is essential to perform initial configuration steps to ensure the environment is secure, functional, and ready for use. Refer to Post-installation configuration in this guide.

5 Initial configuration

After setting up a Boundary cluster, it is essential to perform initial configuration steps to ensure the environment is secure, functional, and ready for use. Below are the prerequisites and high-level steps you should follow.

5.1 Prerequisites

Boundary

- You have reviewed the Boundary deployment guide and have a running Boundary cluster and the database has been initialized.
- You have a valid recovery KMS as defined in the controller configuration file.

HCP Boundary

- You have reviewed the HCP Boundary setup instructions and have a running HCP Boundary cluster.
- You have authenticated to the HCP Boundary cluster using your administrator credential.

5.2 High-level steps for initial configuration

> **Note**
>
> Steps 1-6 listed here apply only to the Boundary Enterprise version. For HCP Boundary, the system pre-creates the password-based authentication method, roles for global administrator and anonymous users, and the global administrator credentials during the HCP Boundary cluster setup.Steps 7-9 apply to both HCP Boundary and Boundary Enterprise. The term *Boundary* applies to both products for those steps.Run the commands below against the Boundary cluster. You can specify the Boundary cluster address using the `-addr` flag with the command or by exporting the `BOUNDARY_ADDR` environment variable before executing the commands.For example, `export BOUNDARY_ADDR=<HCP-BOUNDARY-CLUSTER-URL|BOUNDARY-ENT-CLUSTER-URL>`

1. Authenticate with recovery KMS
    - Start by authenticating to Boundary using the recovery KMS via the command line. This grants you superuser privileges to configure initial resources. Please refer to log in with recovery KMS for detailed instructions.
2. Create an initial password auth method
    - Set up a password-based authentication method at the global scope level. Use this method only for the initial setup, testing, or as a fallback mechanism if the primary auth method (for example LDAP or OIDC) fails or is unavailable.
    - For example, run the below command to create a password auth method at the global scope level.

```
boundary auth-methods create password \
  -recovery-config /tmp/recovery.hcl \
  -scope-id 'global' \
  -name 'password' \
  -description 'Password auth method'
```

3. Create a login administrator account
    - Create a login account within the password authentication method. Use this account to manage and configure Boundary. Please refer to create login account for the command to create the login account.

- For example, run the below command to create a login account.

```
boundary accounts create password \
  -recovery-config /tmp/recovery.hcl \
  -login-name "admin" \
  -auth-method-id <auth_method_id_from_last_step>
```

4. Create an administrator user and associate with the login account

   - Create the user `admin_user` and link it to the login account created in the previous step. Configure this user with administrative privileges to manage Boundary resources in the next step.
   - Run the commands below to create and associate the user to the login account.

```
boundary users create -scope-id 'global' \
  -recovery-config /tmp/recovery.hcl \
  -name "admin_user" \
  -description "Global admin user"

boundary users add-accounts \
  -recovery-config /tmp/recovery.hcl \
  -id <admin_user_id> \
  -account <admin_account_id>
```

5. Create an administrator role and assign to administrator user

   - Define an administrator role with permissions to manage all resources in Boundary. Assign this role to the admin user.
   - Run the below commands to create and assign the administrator role to the administrator user.

```
boundary roles create -name 'global_admin' \
  -recovery-config /tmp/recovery.hcl \
  -scope-id 'global'

boundary roles add-grants -id <global_admin_role_id> \
  -recovery-config /tmp/recovery.hcl \
  -grant 'ids=*;type=*;actions=*'

boundary roles add-principals -id <global_admin_role_id> \
  -recovery-config /tmp/recovery.hcl \
  -principal '<admin_user_id>'
```

6. Create a role for anonymous (unauthenticated) users

   - Run below command to allow anonymous users to list scopes and auth methods in the global and organization scopes.

```
boundary roles create -name 'global_anon_listing' \
  -recovery-config /tmp/recovery.hcl \
  -scope-id 'global

boundary roles add-grants -id <global_anon_listing_id> \
  -recovery-config /tmp/recovery.hcl \
  -grant 'ids=*;type=auth-method;actions=list,authenticate' \
  -grant 'ids=*;type=scope;actions=list,no-op' \
  -grant 'ids={{.Account.Id}};actions=read,change-password'

boundary roles add-grant-scopes -id <global_anon_listing_id> -grant-scope-id "
    children"

boundary roles add-principals -id <global_anon_listing_id> \
  -recovery-config /tmp/recovery.hcl \
  -principal 'u_anon'
```

7. Login using password auth method

   - Run below command to login using password auth method.

```
boundary authenticate password \
  -auth-method-id <auth_method_id>
```

8. Create an Organization and Project scope

   - Set up an Organization and a Project scope to organize resources and manage access control within Boundary.
   - Please refer to create organization and project scope for detailed instructions.

9. Create roles with administrative privileges at the Organization and Project scope levels.

   - Run below command to create organization administrator role.

```
boundary roles create -name 'org_admin' \
  -scope-id 'global'

boundary roles set-grant-scopes \
  -id <org_admin_id> \
  -grant-scope-id <org_scope_id>

boundary roles add-grants -id <org_admin_id> \
  -grant 'ids=*;type=*;actions=*'
```

- Run below command to create project administrator role.

```
boundary roles create -name 'project_admin' \
  -scope-id <org_scope_id> \
  -grant-scope-id <project_scope_id>

boundary roles add-grants -id <project_admin_id> \
  -grant 'ids=*;type=*;actions=*'
```

- Assign users, groups, or managed groups as principals to the organization administrator and project administrator roles after setting up the LDAP or OIDC auth method. Please refer to the respective sections in this document for the instructions.
- Run below commands to assign a principal (user, group, or managed group) to the organization administrator or project administrator roles.

```
boundary roles add-principals -id <org_admin_id> \
  -principal <user_id|group_id|managed_group_id>

boundary roles add-principals -id <project_admin_id> \
  -principal <user_id|group_id|managed_group_id>
```

6 Secure proxy

Secure proxy is a Boundary worker capability that is responsible for proxying sessions between clients and targets.

6.1 Operating modes

The following sections describe the various operating modes, topology, and network connectivity requirements.

- Proxying target sessions
- Proxying Vault connections
- Proxying multi-hop sessions

The mode of operation classes Boundary workers per the following.

- Ingress worker
- Intermediary worker
- Egress worker

All workers use the same binary, and configuration determines their mode of operation. The access requirements determine which operating modes satisfy the topology. For example, proxying multi-hop sessions require a minimum of ingress and egress workers.

All operating modes and topologies apply to HCP Boundary and Boundary Enterprise.

6.1.1 HCP Boundary

Self-managed workers allow users to connect to private endpoints securely without routing traffic through HCP Boundary or HashiCorp-managed infrastructure.

For example, organizations with on-premise environments can deploy self-managed workers in their private networks to allow connectivity to those targets with HCP Boundary, often in a multi-hop topology with ingress and egress workers.

Self-managed workers use public key infrastructure (PKI) for authentication. They authenticate to Boundary using a certificate-based method that allows you to deploy workers without using a shared key management service (KMS).

6.1.2 Proxying target sessions

Achieve proxying sessions between clients and targets with a single-layered approach, which is the minimal requirement for establishing sessions. This approach is a simplified topology - use it only in non-production scenarios such as testing or proof-of-concepts.

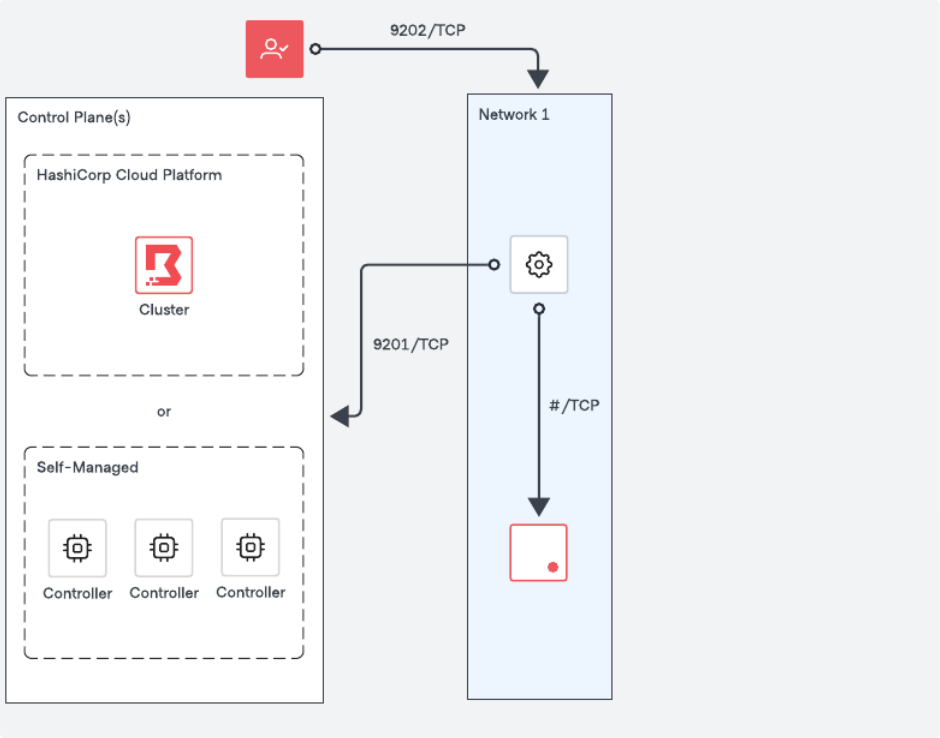


Figure 5: Boundary Proxy Target Sessions

Network requirements

- Outbound connectivity (default port TCP-9201) to an existing trusted Boundary control point, for example a Boundary worker or the Boundary control plane, or in other words, the cluster URL.
- Outbound connectivity to the remote service port of the target.
- Inbound connectivity (default port TCP-9202) from clients establishing sessions to the targets

6.1.3 Proxying vault connections

Similar to Proxying target sessions, a worker can proxy connections to private Vault clusters to provide integration between Boundary and Vault.

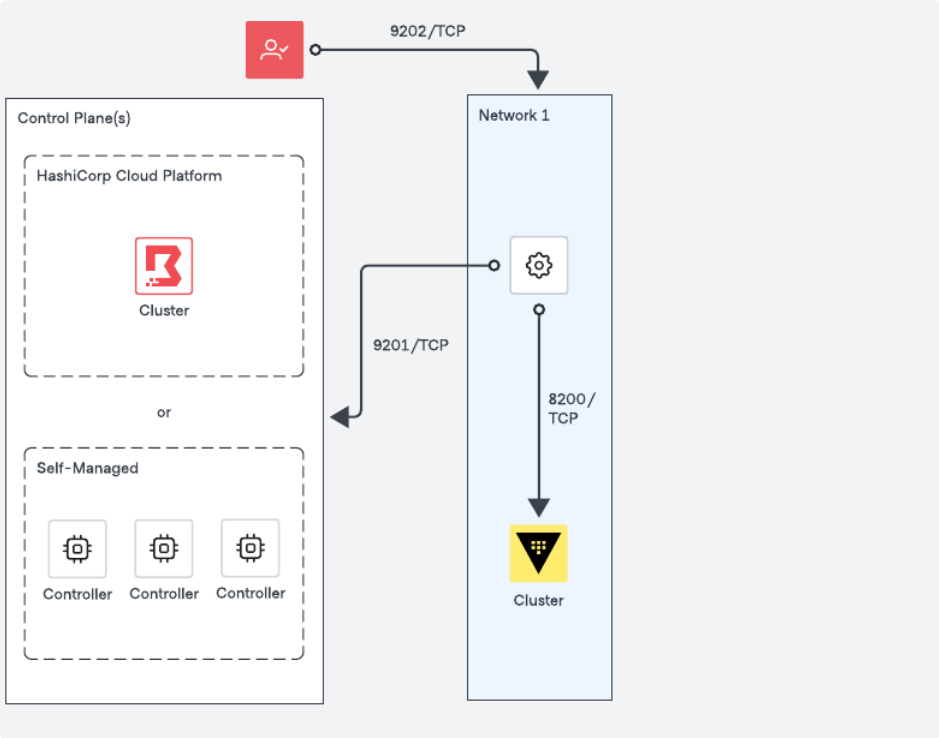


Figure 6: Boundary Proxy Vault Connections

Network requirements

- Outbound connectivity (default port TCP-9201) to an existing trusted Boundary control point, for example a Boundary worker or the Boundary control plane, or in other words, the cluster URL.
- Outbound connectivity (default port TCP-8200) to the Vault cluster.

6.2 High availability and sizing

6.2.1 Recommendations for high availability

Each network enclave that Boundary accesses should have at least one worker to provide connectivity. Deploying workers into each network enclave enables organizations to minimize and simplify the firewall/security requirements typically required to provide connectivity to targets.

We recommend at least three workers per network enclave to ensure high availability for production environments. Worker session assignment is intelligently dictated by the Boundary control plane based on the following:

- Which workers are candidates to proxy a session based on the worker's tags and the target's worker filter, and
- The health and connectivity of candidate workers, you **do not need** a load balancer to manage worker traffic.

The constraints of your access use case and the sensitivity of workloads in each network enclave dictate the redundancy and size you require for your workers.

6.2.2 Sizing guidance

We have divided sizing recommendations into two cluster sizes, and they represent an initial starting point based on the use case(s).

- **Small** clusters are appropriate for most initial production deployments or non-production environments.
- **Large** clusters are production environments with a large number of Boundary clients.

We recommend monitoring your cloud providers' network throughput limitations continuously for your machine types as you use Boundary and observe relevant metrics where possible, in addition to other host metrics, so that you can scale Boundary horizontally or vertically as needed.

Provider	Size	Instance/VM Type
AWS	Small	m5.large, m5.xlarge
	Large	m5n.2xlarge, m5n.4xlarge
Azure	Small	Standard_D2s_v3, Standard_D4s_v3
	Large	Standard_D8s_v3, Standard_D16s_v3
GCP	Small	n2-standard-2, n2-standard-4
	Large	n2-standard-8, n2-standard-16

Additional documentation

- AWS: EC2 Network Performance and Monitoring EC2 Network Performance
- Azure: Azure Virtual Machine Throughput and Accelerated Network for Azure VMs
- GCP: Network Bandwidth and About Machine Families

Worker performance

Performance is most affected by the number of concurrent sessions the worker is proxying and the data transfer rates within those sessions. The size of workers is dependent on how you use Boundary, for example, if you use SSH connections and HTTP access, your instance selection and performance differ from a large data transfer use case.

Organizations should monitor worker performance using the appropriate telemetry platform. In addition to the typical OS metrics (for example CPU, Memory, Disk, Network), Capture Boundary-specific worker metrics to provide better insight into performance, and enable Boundary administrators to make decisions about scaling.

7 Accessing private resources

7.1 Multi-hop sessions

Organizations with complex network topologies often require inbound traffic to route through multiple network enclaves to reach the target system.

Even in complex networks with strict outbound-only policies, Multi-hop sessions allow you to chain two or more workers across numerous networks to form reverse proxy connections between the user and the target.

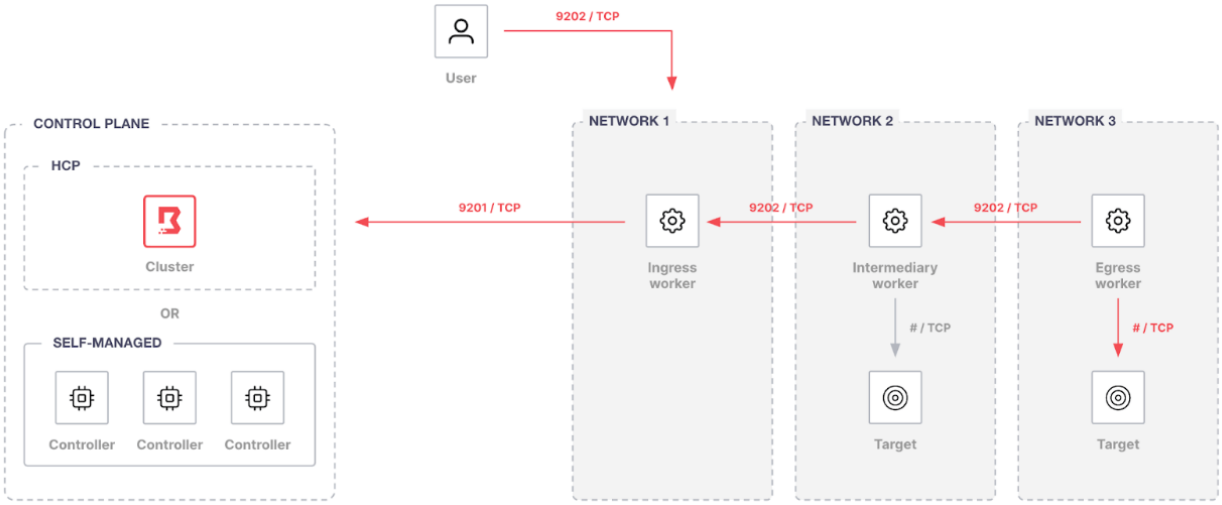


Figure 7: Multi-hop Sessions

When workers operate as part of a multi-hop chain, they have three distinct functions:

7.1.1 Ingress worker

An ingress worker is accessible by Boundary users and typically deployed on edge or public networks.

Network requirements

1. Outbound connectivity (TCP-9201) to an existing trusted Boundary control point, e.g., a Boundary worker or the Boundary control plane, or in other words, the cluster URL.
2. Inbound connectivity (TCP-9202) from users establishing sessions

7.1.2 Intermediary worker

An intermediary worker is optional and is deployed between ingress and egress workers.

Network requirements

1. Outbound connectivity (TCP-9202) to an upstream worker. An upstream worker may be an ingress worker or another intermediary worker. Any upstream or intermediary worker must eventually connect to an ingress worker.

2. Inbound connectivity (TCP-9202) from a downstream worker. A downstream worker may be an egress worker or another downstream worker. Any downstream or intermediate worker must eventually connect to an egress worker.

7.1.3 Egress worker

An egress worker provides connectivity to the target. The worker initiates reverse proxy connections to the intermediary or ingress workers.

Network requirements

1. Outbound connectivity (TCP-9202) to an upstream worker
2. Outbound connectivity (client target port) to the target.

7.1.4 Redundancy

A recommendation is to create redundant or multiple ingress paths to ensure elasticity. Typically, a Boundary architecture *might* include multiple ingress paths to a particular target. By providing multiple paths to a target, worker ingress is moved away from a single point of failure, as workers are not highly available at this time.

7.2 References

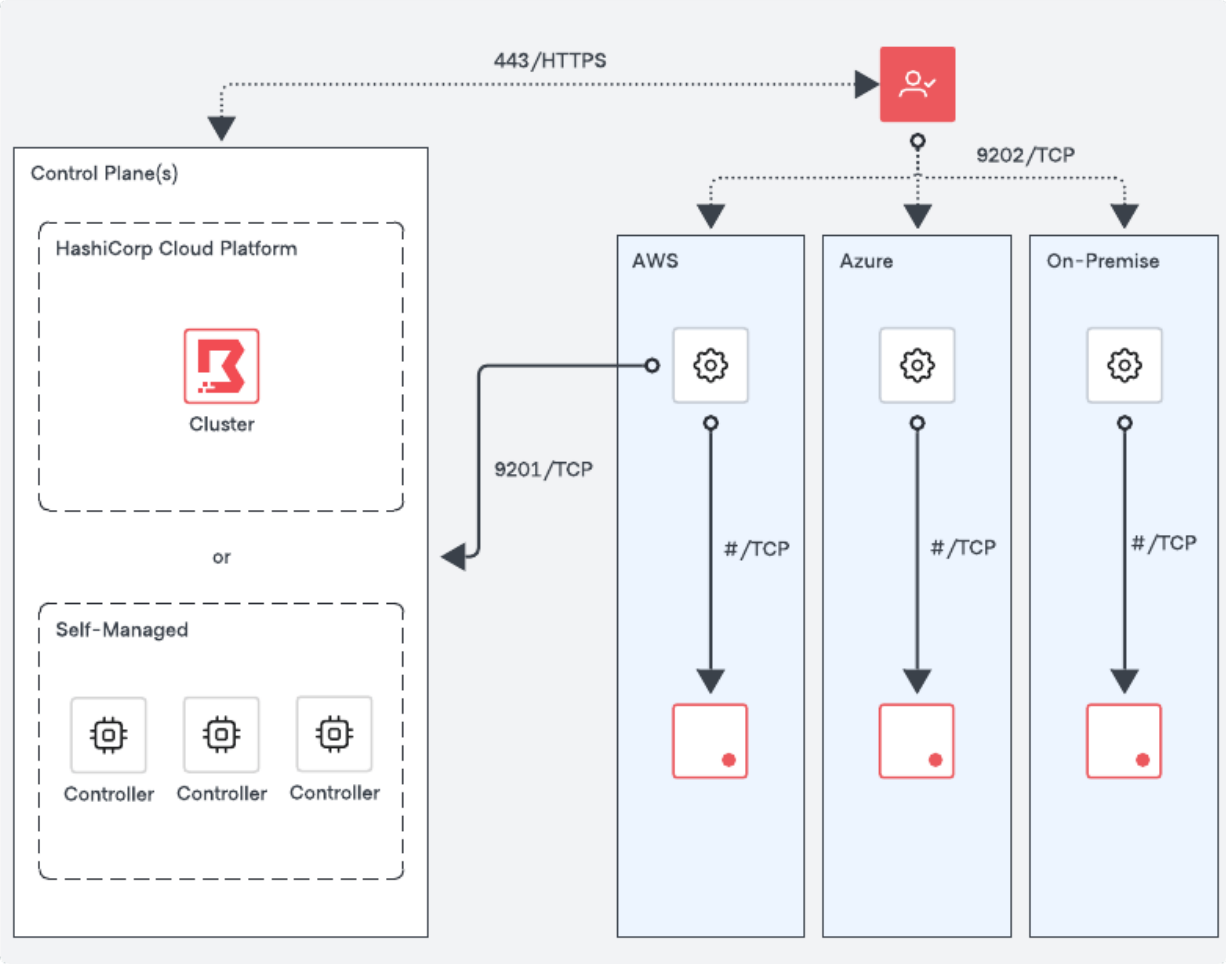
Multi-hop sessions with HCP Boundary tutorial shows an example how targets can be configured to be accessed through the chain of Boundary workers.

8 Worker aware targets

In traditional multi-datacenter and multi-cloud operating models, it's common to deploy a control plane for each environment, complete with controllers and workers, to minimize latency or meet security standards. However, managing multiple controllers can increase complexity, costs and operational overhead.

Boundary's control plane uses worker tags and filters to coordinate which workers can handle a target's session. A typical example is allowing a single set of controllers to live in

one environment and placing workers in many other environments where their proxy targets live.



8.1 Multi-region deployments

This section provides an example of a multi-region deployment of HCP Boundary across multi-datacenter and multi-cloud environments. The same concept can be applied to self-managed Boundary Enterprise.

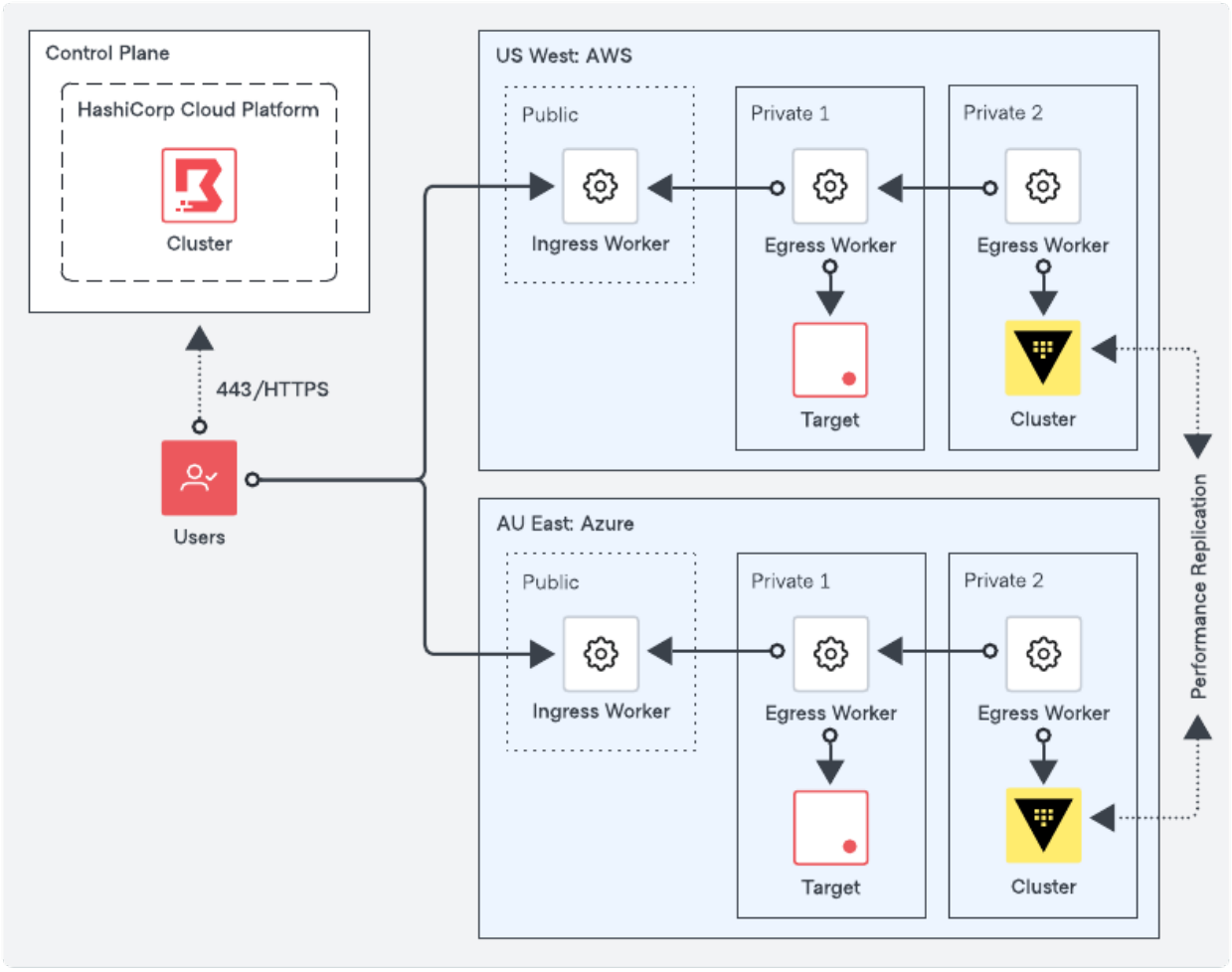


Figure 8: Multi-Region Deployment

- There is a publicly accessible ingress worker at each region's edge for user connectivity. The user session would transit through the appropriate ingress worker based on the target based on the target.
- Multi-hop sessions, where egress workers would proxy from one another to simplify connectivity requirements.
- Vault deployment in each region to comply with availability and regulatory requirements. The Vault cluster resides in a secure private network.
- Session recording data is stored locally within each region.

9 Changelog

This page records notable changes to the Boundary Installation Guide.

9.1 2026-08

- Restructured legacy Boundary HVD content into the HVD 2.0 Boundary Installation Guide as part of the Installation / Administration / User guide migration.

10 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Andrei Burd
- Danny Knights
- Jeff Mitchell
- Jim Lambert
- Mark Lewis
- Nick Wong
- Ravi Panchal
- Sai Linn Thu
- Shriram Rajaraman
- Theodore Marx
- Tony Phan
- Van Phan